

Cell 4 — Three Preprocessing Variants (Notebook cell index: 8)

This cell defines stopword sets, stemmer/lemmatizer objects, three preprocessor functions, and the PREPROCESSORS dictionary.

Granular line-by-line breakdown

Shared NLP objects

```
STOPWORDS = set(stopwords.words('english'))
```

- `stopwords.words('english')` — NLTK returns a **list** of common English stopwords.
- `set(...)` — Converts list to a **set** for **O(1)** membership tests (`word in STOPWORDS`).
Why: Many token checks per document; sets are faster than lists here.

```
NEGATION_KEEP = {'no', 'not', 'nor'}
```

- `{...}` — Set literal of words to **never** remove in the “optimum” path.
- **Why:** Negations change meaning; dropping them can confuse classifiers.

```
STOPWORDS_OPTIMUM = STOPWORDS - NEGATION_KEEP
```

- `-` — Set **difference**: all English stopwords **except** the negation trio.
- `STOPWORDS_OPTIMUM` — Stoplist used in `preprocess_optimum` only.

```
stemmer = PorterStemmer()  
lemmatizer = WordNetLemmatizer()
```

- `PorterStemmer()` — Constructs stemmer instance; `stemmer.stem(word)` strips suffixes aggressively.
 - `WordNetLemmatizer()` — Constructs lemmatizer; `lemmatizer.lemmatize(word)` returns dictionary base form (with default POS = noun).
-

preprocess_none

```
def preprocess_none(text):  
    return str(text)
```

- `def preprocess_none(text):` — Function taking one headline text.
 - `return str(text)` — Converts input to string and returns **unchanged** content (only type coercion). **Why:** “No preprocessing” baseline but still safe if a cell is non-string.
-

normalize_text

```
def normalize_text(text):  
    text = str(text)
```

- Ensures string type before regex/HTML ops.

```
    text = html.unescape(text)
```

- Decodes HTML entities to raw characters.

```
    text = re.sub(r'<[>]+>', ' ', text)
```

- `re.sub` — Replace HTML tags with a single space.

```
text = re.sub(r'http\S+|www\S+', ' ', text)
```

- `http\S+` — http followed by non-whitespace (URL body).
- `www\S+` — URLs starting with www.
- **Why:** Remove URL tokens that inflate vocabulary.

```
text = re.sub(r'^a-zA-Z\s', ' ', text)
```

- **Character class** `[^a-zA-Z\s]` — Anything **not** letters or whitespace → replaced by space. **Why:** Strip digits/punctuation for a letters-only token stream.

```
text = re.sub(r'\s+', ' ', text).strip().lower()
```

- `r'\s+'` — Collapse whitespace.
- `.strip()` — Trim ends.
- `.lower()` — Case folding for fewer duplicate tokens.

```
return text
```

- Returns the normalized single string (still **space-separated words**, not a list).

preprocess_extreme

```
def preprocess_extreme(text):
    text = normalize_text(text)
```

- `text` — Reuses name: now holds normalized string.

```
tokens = text.split()
```

- `.split()` — Default split on whitespace → **Python list** of word strings.

```
tokens = [t for t in tokens if t not in STOPWORDS and len(t) > 1]
```

- **List comprehension** — Keeps token `t` only if not a stopword and length `> 1` (drops single-letter noise).

```
tokens = [lemmatizer.lemmatize(t) for t in tokens]
```

- Applies WordNet lemmatization to each surviving token.

```
tokens = [stemmer.stem(t) for t in tokens]
```

- Applies Porter stemming **after** lemmatization (very aggressive normalization).

```
return ' '.join(tokens)
```

- `' '.join(list)` — Joins tokens back into one string with spaces (models expect one string per document).

preprocess_optimum

```
def preprocess_optimum(text):
    text = normalize_text(text)
    tokens = text.split()
    tokens = [t for t in tokens if t not in STOPWORDS_OPTIMUM and len(t) >
```

- Same as extreme **except** `STOPWORDS_OPTIMUM` keeps `no/not/nor`.

```
tokens = [lemmatizer.lemmatize(t) for t in tokens]
return ' '.join(tokens)
```

- Lemmatize only—**no** `stemmer.stem`, preserving slightly more surface form than

extreme.

Registry dict

```
PREPROCESSORS = {  
    'none': preprocess_none,  
    'extreme': preprocess_extreme,  
    'optimum': preprocess_optimum  
}
```

- 'none' — String **key** (human-readable name for logs/plots).
 - `preprocess_none` — **Function object** (no `()`): stored by reference for later `prep_fn(row)`.
 - **Why dict:** Later code does `for prep_name, prep_fn in PREPROCESSORS.items()`.
-

Mechanism of the whole cell

Build shared NLP resources → define three functions from raw string → normalized/aggressive/moderate string → expose them in one dictionary for loops later.

Role and importance in the notebook

Central switchboard for the comparative study: every model loop selects `prep_fn` from here so preprocessing is the controlled variable.